Actividad 4. Algoritmo por K-Medias.

Primero se cargó el archivo “Iris.csv”, la base contiene 150 observaciones y cinco columnas.

```
[21]: import pandas as pd
      import matplotlib.pyplot as mp
      import seaborn as ab
      import os

      os.chdir("D:/gus6e/Documents")

      from sklearn.cluster import KMeans

      dataset = pd.read_csv("iris.csv")
      dataset.head()
```

[21]:

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa

Primero se aplica el método del codo, en este procedimiento se prueba el algoritmo con distintas cantidades de clusters y se observa cómo cambia la distancia de las observaciones respecto al centro de su grupo. Mientras se agregan más clusters, esa distancia normalmente disminuye; lo que interesa es identificar el momento en que la reducción deja de ser tan grande. En esta primera parte se las primeras cuatro columnas para crear los campos y se quitan las alertas por la antigüedad del código.

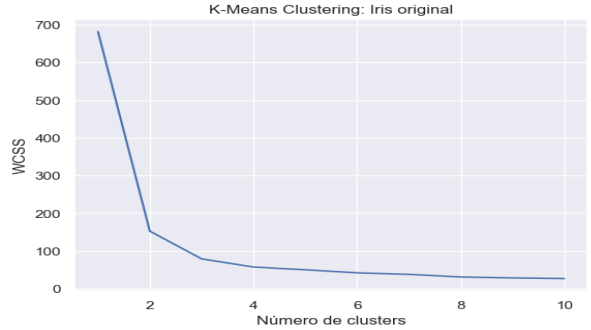
```
[22]: fields = dataset.iloc[:,[0,1,2,3]].values
      print(fields)
```

```
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]
 [5.4 3.9 1.7 0.4]
 [4.6 3.4 1.4 0.3]
 [5.  3.4 1.5 0.2]
 [4.4 2.9 1.4 0.2]
 [4.9 3.1 1.5 0.1]
 [5.4 3.7 1.5 0.2]
 [4.8 3.4 1.6 0.2]
 [4.8 3.  1.4 0.1]
 [4.3 3.  1.1 0.1]
 [5.8 4.  1.2 0.2]
 [5.7 4.4 1.5 0.4]
 [5.4 3.9 1.3 0.4]
 [5.1 3.5 1.4 0.3]
```

```
[23]: import warnings
      warnings.filterwarnings("ignore")
```

En la gráfica de los datos originales se observa que la disminución es marcada al principio y que, después de tres clusters, la curva empieza a bajar de una manera más gradual. Por eso se eligieron tres grupos de acuerdo con el criterio del codo.

```
#Graficación del "Codo de Jambu"
ab.set()
mp.plot(range(1,11), wcss)
mp.title("K-Means Clustering: Iris original")
mp.xlabel("Número de clusters")
mp.ylabel("WCSS") #Weighted cluster sum of squares
mp.show()
```



Después se aplica el algoritmo de K-Medias con tres clusters. El algoritmo asignó cada flor a uno de los tres grupos y calculó sus centroides. Posteriormente se hace una gráfica de dispersión en la que cada color representa un grupo y los puntos negros señalan los centroides. En la gráfica se utilizan el largo y el ancho del pétalo.

Al observar los puntos se distingue un grupo bastante separado de los otros, mientras que los dos grupos restantes se encuentran más próximos entre sí. Esto permite ver que algunas flores tienen características más claramente diferenciadas y otras se parecen más.

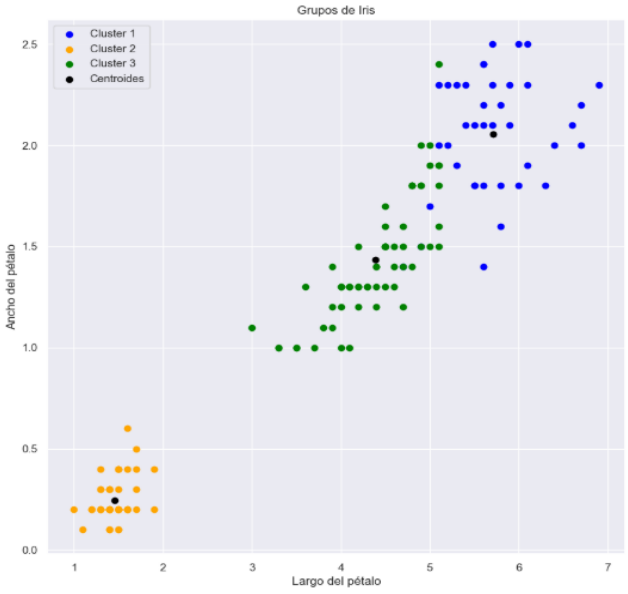
```
[26]: kmeans = KMeans(n_clusters = 3, init = "k-means++", random_state = 42)
cluster_values = kmeans.fit_predict(fields)
print(cluster_values)
cluster_values_original = cluster_values.copy()
```

```
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 2 0 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 2 2 2 0 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 0 2 0 0 0 0 2 0 0 0 0
 0 0 2 2 0 0 0 0 2 0 2 0 2 0 0 2 2 0 0 0 0 0 2 0 0 0 0 2 0 0 0 2 0 0 0 2 0 2 0
 0 2]
```

```
[27]: mp.figure(figsize = (10,10))
mp.scatter(fields[cluster_values==0,2], fields[cluster_values==0,3], c = "blue", label = "Cluster 1")
mp.scatter(fields[cluster_values==1,2], fields[cluster_values==1,3], c = "orange", label = "Cluster 2")
mp.scatter(fields[cluster_values==2,2], fields[cluster_values==2,3], c = "green", label = "Cluster 3")

mp.scatter(kmeans.cluster_centers_[:,2], kmeans.cluster_centers_[:,3], c = "black", label = "Centroides")

mp.title("Grupos de Iris")
mp.xlabel("Largo del pétalo")
mp.ylabel("Ancho del pétalo")
mp.legend()
mp.show()
```



Además del método del codo, se usa el criterio de silueta, que sirve para evaluar qué tan bien agrupadas quedaron las observaciones, o bien, más cerca de las de su propio grupo y mejor separadas de las que pertenecen a otros grupos.

Con los datos originales, el valor más alto fue aproximadamente 0.6810 al utilizar dos clusters. En cambio, con tres clusters el valor fue aproximadamente 0.5512. Por lo tanto, el criterio de silueta favoreció dos grupos, mientras que el método del codo sugirió tres.. Para continuar y poder comparar los dos análisis utilicé tres clusters, que fue la cantidad seleccionada a partir de la gráfica del codo.

```
[28]: # Criterio de Silueta ("Silhouette")
from sklearn import datasets
from sklearn.metrics import silhouette_score

[29]: X = dataset.iloc[:, [0,1,2,3]].to_numpy()
for j in range(2,12):
    kmeans = KMeans(n_clusters = j, random_state = 42)
    kmeans.fit_predict(X)
    #Calculamos el Score de Silueta
    score = silhouette_score(X,kmeans.labels_,metric = "euclidean")
    print("Score Silhouette:", "K =", j, "Score =", score)

Score Silhouette: K = 2 Score = 0.6810461692117465
Score Silhouette: K = 3 Score = 0.5511916046195927
Score Silhouette: K = 4 Score = 0.4976433179321929 6
Score Silhouette: K = 5 Score = 0.4930804067193 53
Score Silhouette: K = 6 Score = 0.3678464984712254
Score Silhouette: K = 7 Score = 0.3542978877198859
Score Silhouette: K = 8 Score = 0.3446797218056206
Score Silhouette: K = 9 Score = 0.3155887853389787
Score Silhouette: K = 10 Score = 0.30141437453251546
Score Silhouette: K = 11 Score = 0.2687356216412038

[30]: # Algoritmo KMeans aplicado a cuatro columnas de nuestra base original
# Bajo una transformación de PCA

fields2 = dataset.iloc[: , [0,1,2,3]].values
print(fields2)

[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]
 [5.4 3.9 1.7 0.4]
 [4.6 3.4 1.4 0.3]
 [5.  3.4 1.5 0.2]
 [4.4 2.9 1.4 0.2]
 [4.9 3.1 1.5 0.1]
 [5.4 3.7 1.5 0.2]
 [4.8 3.4 1.6 0.2]
 [4.8 3.  1.4 0.1]
 [4.3 3.  1.1 0.1]
 [5.8 4.  1.2 0.2]
 [5.7 4.4 1.5 0.4]
 [5.4 3.9 1.3 0.4]
 [5.1 3.5 1.4 0.3]
```

En la segunda parte se repiten los códigos, pero aplicando PCA antes, que es una técnica de reducción de dimensiones ya vista en actividades anteriores. Lo que hace es resumir las variables originales en nuevas componentes que conservan la mayor parte de las diferencias presentes en los datos.

```
[31]: from sklearn import decomposition

pca = decomposition.PCA(n_components=2)
pca.fit(fields2)
fields2 = pca.transform(fields2)
fields2

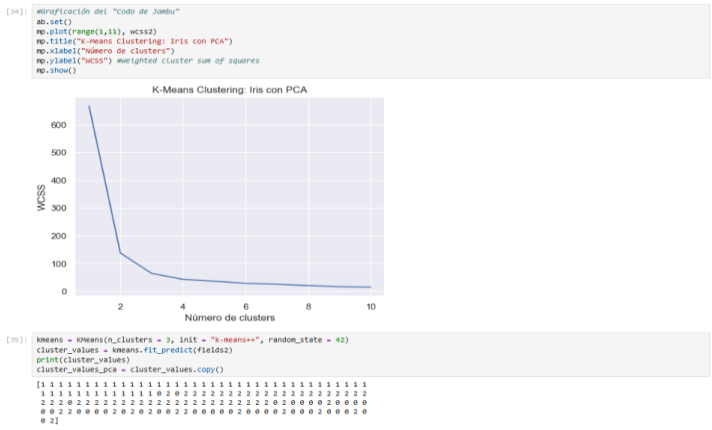
[31]: array([[-2.68412563,  0.31939725],
       [-2.71414169, -0.17700123],
       [-2.88899057, -0.14494943],
       [-2.74534286, -0.31829898],
       [-2.72871654,  0.32675451],
       [-2.28085963,  0.74133045],
       [-2.82053775, -0.08946138],
       [-2.62614497,  0.16338496],
       [-2.88638273, -0.57831175],
       [-2.6727558 , -0.11377425],
       [-2.50694709,  0.6450689 ],
       [-2.61275523,  0.01472994],
       [-2.78610927, -0.235112  ],
       [-3.22380374, -0.51139459],
       [-2.64475039,  1.17876464],
       [-2.38603903,  1.33806233],
       [-2.62352788,  0.81067951],
       [-2.64829671,  0.31184914],

[32]: print("Varianza explicada por cada componente:", pca.explained_variance_ratio_)
print("Varianza total conservada:", pca.explained_variance_ratio_.sum())

Varianza explicada por cada componente: [0.92461872 0.05306648]
Varianza total conservada: 0.9776852063187922
```

En lugar de trabajar con cuatro columnas, se pasó a trabajar con dos componentes principales. El resultado mostró que la primera componente conservó aproximadamente el 92.46 % de la variabilidad y la segunda alrededor del 5.31 %. En conjunto conservaron cerca del 97.77 %. Esto significa que, a pesar de reducir las cuatro medidas a solo dos componentes, se mantuvo casi toda la variabilidad que había en la base original. La reducción no equivale a conservar absolutamente toda la información, pero en este caso la pérdida fue pequeña.

Una vez teniendo los dos componentes, se realiza el método del codo nuevamente, siguiendo el mismo procedimiento de la primera parte. En esta nueva gráfica también se observa que la curva cambia de manera importante alrededor de los tres clusters. Por esta razón, el criterio del codo volvió a sugerir la formación de tres grupos, incluso después de reducir las dimensiones de la base.



Se aplica K-Medias a los datos transformados, nuevamente con tres clusters. Cada punto representa una flor, los colores indican el grupo al que fue asignada y los puntos negros muestran los centroides. La diferencia respecto a la primera gráfica es que ahora los ejes no corresponden a medidas individuales del pétalo, sino a las dos componentes obtenidas con PCA. Esto hace más sencilla la representación del análisis completo, porque las dos dimensiones que aparecen en la gráfica resumen la información de las cuatro variables originales.



También calculé criterio de silueta con los datos reducidos. El valor mayor fue aproximadamente 0.7057 para dos clusters, mientras que con tres clusters se obtuvo alrededor de 0.5977. De nuevo, este indicador favoreció dos grupos y el codo sugirió tres. Si comparamos los resultados de tres clusters, el criterio de silueta aumentó de aproximadamente 0.5512 en los datos originales a 0.5977 con PCA, lo que señala una mejor separación relativa de los grupos en el espacio de las dos componentes.

```
X = fields2
for j in range(2,12):
    kmeans = KMeans(n_clusters = j, random_state = 42)
    kmeans.fit_predict(X)
    #Calculamos el Score de Silueta
    score = silhouette_score(X,kmeans.labels_,metric = "euclidean")
    print("Score Silhouette:", "K =", j, "Score =", score)
```

```
Score Silhouette: K = 2 Score = 0.7056703225102109
Score Silhouette: K = 3 Score = 0.5976764219547989
Score Silhouette: K = 4 Score = 0.5577409232246782
Score Silhouette: K = 5 Score = 0.5100407194899204
Score Silhouette: K = 6 Score = 0.40319738737442684
Score Silhouette: K = 7 Score = 0.3868263997713609
Score Silhouette: K = 8 Score = 0.4407603324965402
Score Silhouette: K = 9 Score = 0.4020143876366364
Score Silhouette: K = 10 Score = 0.41588658017532526
Score Silhouette: K = 11 Score = 0.3994325519305243
```

Preguntas

¿Se obtuvieron los mismos resultados de agrupamiento?

Al comparar las asignaciones de las 150 flores con tres clusters, cada una quedó en el mismo grupo antes y después de aplicar PCA. También se conservaron los mismos tamaños de grupos: uno quedó integrado por 39 flores, otro por 50 y el tercero por 61. Por lo tanto, en este ejercicio la reducción de dimensiones no modificó el resultado de agrupación obtenido con K-Medias para tres clusters, entonces sí se obtienen los mismos resultados.

¿Qué ventaja práctica tuvo utilizar PCA antes de K-Medias?

La principal ventaja es que se puede trabajar y graficar los resultados usando solo dos dimensiones, sin perder gran parte de la variabilidad de las cuatro características originales. En este caso, PCA resultó especialmente útil porque conservó aproximadamente el 97.77 % de la variabilidad y, al mismo tiempo, dejó intactas las agrupaciones obtenidas con tres clusters.